VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



REPORT
CAPSTONE PROJECT
SEMESTER 261 ACADEMIC YEAR 2026-2027

INTELLIGENT ONLINE LEARNING MANAGEMENT SYSTEM INTEGRATING AI

MAJOR: COMPUTER SCIENCE

COUNCIL: <NAME>
SUPERVISOR(s): Dr. Truong Tuan Anh
REVIEWER:

——o0o——

STUDENT 1: Dinh Cao Thien Loc - 2352708
STUDENT 2: Tran Tri Dung - 2252133
STUDENT 3: Thai Thien Hai Long - 2352699
STUDENT 4: Nguyen Manh Tri Minh - 2352752

HO CHI MINH CITY, December 2026

Instructor’s Signature

_____ Date: _____

<NAME> (Instructor)

...

Faculty of Computer Science and Engineering

Declaration of Authenticity

We declare that we solely conducted this specialized project, under the supervision of <NAME> at the Faculty of Computer Science and Engineering, Vietnam National University - Ho Chi Minh City University of Technology.

We have taken care to properly acknowledge and document all external sources and references used in the project.

If there is any instance of plagiarism, we are ready to accept the consequences. Ho Chi Minh City University of Technology - Vietnam National University HCMC will not be held responsible for any copyright violations that may have occurred during my research.

Ho Chi Minh City, December 2024

Authors,

Acknowledgement

We would like to express my appreciation to <NAME> for his invaluable guidance. Our research has greatly benefited from his deep knowledge, perceptive criticism, and constant support.

Besides, we are grateful for the help from ... for valuable comments for research aspect and our proposed pipeline, including pointing out strengths as well as weaknesses of previous works on multimodal models so that we can elaborate on our ideas more effectively.

In addition, we would like to extend our gratitude to my family. Their unwavering faith in our abilities and constant encouragement have been my pillars of strength. Their belief in my potential has been a constant source of motivation and resilience. We are forever grateful for their love and support.

Abstract

Table of Glossary

Contents

1 Introduction **1**

1.1 Background 1

1.2 Goals 2

1.2.1 Practical Significance 2

1.2.2 Scientific and Technical Significance 3

1.2.3 Significance for the Project Team 3

1.2.4 General Objective 3

1.2.5 Specific Objectives 4

1.3 Scope 4

1.3.1 Scope 4

1.3.2 Limitations 5

1.4 Thesis Structure 6

2 Theoretical Background **7**

2.1 Related Systems Survey 7

2.1.1 Canvas LMS 7

2.1.2 D2L Brightspace 9

2.2 Theory Foundation 12

2.2.1 Webserver 12

2.2.2 Transaction Script pattern 12

2.2.3 SQL 12

2.2.4 ERD (Chen Notation ERD) 13

2.3 Technologies 14

2.3.1 Python and FastAPI 14

2.3.2 Uvicorn . . . 15
2.3.3 PostgreSQL . . . 15
2.3.4 Nginx . . . 16
2.3.5 Docker . . . 16
2.3.6 PHP . . . 17
2.3.7 Moodle . . . 17
2.3.8 Webpack . . . 19
2.3.9 React App . . . 19
2.3.10 Generative AI . . . 19
2.3.11 GitHub . . . 20

3 System Analysis **21**
3.1 Functional Requirements . . . 21
3.1.1 Common user functions . . . 21
3.1.2 Student module . . . 22
3.1.3 Teacher module . . . 23
3.1.4 Administrator module . . . 24
3.1.5 Data integration and document processing . . . 25
3.1.6 AI learning assistant (RAG chatbot) . . . 26
3.1.7 Academic early warning . . . 27
3.1.8 Course recommendation . . . 29
3.2 Non-functional Requirements . . . 29
3.2.1 Performance and scalability . . . 29
3.2.2 Reliability and availability . . . 30
3.2.3 Security and privacy . . . 31
3.2.4 Maintainability, extensibility, and portability . . . 32
3.2.5 Usability and compatibility . . . 33
3.2.6 Quality of AI components . . . 34
3.3 Use case . . . 35
3.3.1 System Actors . . . 35
3.3.2 System use case diagram . . . 37

4 Conclusion **95**

References **95**

List of Tables

2.1.1 Comparative Analysis of Academic LMS Platforms 7
2.1.2 Comparative Analysis of D2L Brightspace 10

2.1.1 Functional requirements: common user functions 21
2.1.2 Functional requirements: student module 22
2.1.3 Functional requirements: teacher module 23
2.1.4 Functional requirements: administrator module 24
2.1.5 Functional requirements: data integration and document processing . . 25
2.1.6 Functional requirements: AI learning assistant 26
2.1.7 Functional requirements: academic early warning 28
2.1.8 Functional requirements: course recommendation 29
2.1.9 Non-functional requirements: performance and scalability 30
2.1.10 Non-functional requirements: reliability and availability 31
2.1.11 Non-functional requirements: security and privacy 31
2.1.12 Non-functional requirements: maintainability, extensibility, and portability 33
2.1.13 Non-functional requirements: usability and compatibility 34
2.1.14 Non-functional requirements: quality of AI components 34
2.1.15 System actors 36

List of Figures

2.1 Entity Type 13
2.2 Weak Entity Type 13
2.3 Attribute 13
2.4 Key Attribute 13
2.5 Partial Key Attribute 13
2.6 Multivalued Attribute 13
2.7 Composite Attribute 13
2.8 Devired Attribute 13
2.9 Relationship Type 14
2.10 Relationship Type of Weak Entity Type 14
2.11 Structural Constraints on Relationship Types 14

3.1 System Use Case Diagram 37
3.2 Student Use Case Diagram 42
3.3 Admin Use Case Diagram 57
3.4 Teacher Use Case Diagram 83

Chapter 1

Introduction

In chapter 1, the overview, objectives, and goals of the research project are illustrated. The outline of the report is also presented.

1.1 Background

In recent decades, the world has been changing rapidly, driven by advances in technology, communication, and the widespread availability of information. Information can now be created, accessed, and shared almost instantly, allowing people to learn about a wide range of fields more easily than ever before. This has made it easier for individuals to gain knowledge and develop new skills in many different areas. However, the rapid expansion of information also creates a new challenge: the amount of knowledge and skills that individuals need to acquire is continuously increasing. As a result, it becomes increasingly difficult for individuals to develop deep expertise in a particular field while keeping up with the growing amount of general knowledge.

To keep up with these changes, technology has become increasingly involved in education, and learning is no longer limited to the traditional classroom. Students can now access learning materials, attend courses, and communicate with educators through online learning systems. Learning management systems (LMSs) have become an important part of higher education, providing a centralized platform for managing courses, learning materials, assignments, and other learning activities. However, while LMSs provide ac-

cess to a large amount of learning information, they do not always provide sufficient support for understanding the different learning situations and needs of individual students. This can make it difficult to provide suitable guidance and timely support throughout the learning process.

With the large amount of learning information collected by LMSs, artificial intelligence (AI) can be used to make better use of this information. Machine learning and deep learning can identify patterns in student learning activities and academic performance that may be difficult to recognize manually. These technologies can help provide a better understanding of students' learning progress and needs. However, the integration of AI into LMSs is still developing, especially in developing countries such as Vietnam. Therefore, there is still a need to explore the application of AI in existing LMSs to provide more appropriate support for both students and teachers.

1.2 Goals

1.2.1 Practical Significance

Learning Management Systems (LMS) such as Moodle are widely adopted by universities to organize courses, assign coursework, administer assessments, and manage grades. However, these systems primarily serve as platforms for storing and managing data and do not yet provide intelligent support to learners and instructors. This project aims to add such capabilities on top of the existing Moodle platform, rather than building a new LMS from scratch. For students, the system allows questions about course content to be posed in natural language and returns answers grounded in the course's own materials, accompanied by source citations. This approach reduces the time students spend searching through large collections of slides, textbooks, and syllabi, and eases the burden on instructors of repeatedly answering the same questions. For instructors, the system provides a tool for the early identification of students at risk of poor academic performance, together with the specific reasons behind each assessment, so that timely reminders and support can be offered instead of problems being discovered only after final grades are released. At the institutional level, the resulting analyses serve as a useful data source

for monitoring learning progress and improving the quality of teaching.

1.2.2 Scientific and Technical Significance

From a technical perspective, the project applies Retrieval-Augmented Generation (RAG) to question answering over academic documents in Vietnamese. This task involves several related challenges, including processing documents in multiple formats, combining semantic and keyword-based retrieval, and controlling the tendency of language models to produce answers that deviate from the source material. In parallel, the project applies machine learning to predict academic risk from students' activity data on the LMS, with particular emphasis on the interpretability of predictions so that non-technical users can understand and trust the results. In addition, building a system composed of multiple components, namely the LMS, backend services, data-processing services, artificial intelligence services, and a user interface, provides an opportunity to practice the entire software development process, from analysis and design through deployment and testing.

1.2.3 Significance for the Project Team

For the project team, this work offers an opportunity to develop teamwork skills within a software engineering process, to practice integrating an existing open-source system with extension components, and to become familiar with artificial intelligence technologies from an applied perspective, aimed at real products rather than purely theoretical research.

1.2.4 General Objective

The general objective of this project is to develop an intelligent learning management system for the university environment, built on the Moodle platform, which integrates artificial intelligence features to support students in their learning and to help instructors monitor and identify early those students at risk of poor academic performance.

1.2.5 Specific Objectives

In terms of business analysis, the team surveys the needs of students, instructors, and administrators in the learning and training management process, and analyzes similar systems such as Moodle, Canvas, and Google Classroom to identify their strengths, weaknesses, and missing functionalities, which form the basis for proposing additional features. On this basis, the functional and non-functional requirements of the system are determined. In terms of technology and design, the team selects appropriate technologies for each component and designs the overall architecture, database, and user interface in a manner that avoids modifying the Moodle core, in order to ensure maintainability and ease of future upgrades. In terms of implementation, the team develops a mechanism for synchronizing data from Moodle, including class lists, students, learning materials, grades, and activity logs, to the extension services. Building on this data, the team develops a chatbot that answers questions based on course materials, with enrollment-based access control and source citations, together with an academic early-warning module that produces a list of at-risk students along with the reasons for each. These two features are delivered to users through interfaces designed for students and instructors. In terms of testing and evaluation, the team verifies the functionality and integration of the system, evaluates chatbot quality in terms of retrieval accuracy and latency, and assesses the quality of the warning model using precision, recall, and F1-score, in order to obtain quantitative evidence on which to base conclusions.

1.3 Scope

1.3.1 Scope

The intended users of the system are students, instructors, and administrators of a university. The core LMS functions, including course management, enrollment, assignments, quizzes, grades, and access control, are inherited from the open-source Moodle platform; this project does not rebuild these functions but instead focuses on extension and integration. The project is carried out in two phases. In the specialized project phase, the team focuses on system analysis and design while implementing the basic features: data

synchronization from Moodle, basic document processing, a minimal chatbot, academic early warning based on threshold rules or a simple model, and a chat interface together with a monitoring dashboard for instructors. In the graduation project phase, the team completes the system and improves the quality of the artificial intelligence components through hybrid retrieval, re-ranking, and an interpretable prediction model, while also optimizing performance and security and carrying out testing, evaluation, and deployment. With respect to artificial intelligence features, the scope of the project covers a chatbot that answers questions based on course materials and an early-warning function for students at risk of poor academic performance. A course recommendation feature for students is planned for the second phase, subject to the available schedule.

1.3.2 Limitations

Because the Moodle core is not modified and only plugins and external services are developed, the features of the system depend on the data and programming interfaces that Moodle provides. Regarding data, since the team does not have access to real learning data from the university, the warning model is built on public datasets or simulated data; consequently, the evaluation results are experimental and do not fully reflect real-world conditions. Regarding documents and language, the chatbot supports common formats such as PDF, DOCX, and PPTX, while low-quality scanned documents, complex tables, and mathematical formulas are handled only to a limited extent. The system prioritizes Vietnamese, and its quality in other languages lies outside the scope of evaluation. Regarding resources, the language model runs on limited hardware and budget, so latency and answer quality may be lower than those of large-scale commercial services. The system is also tested only at a small scale, with a few courses and sample users, and has not been validated at the scale of an entire university. Furthermore, features such as camera-based cheating detection, a dedicated mobile application, and integration with the university's official credit registration system are beyond the scope of this project. Finally, it should be noted that the outputs of the chatbot and the warning model are intended solely as supporting references and do not replace the decisions of instructors.

1.4 Thesis Structure

Chapter 2

Theoretical Background

This chapter presents preliminary knowledge which is fundamental of our project.

2.1 Related Systems Survey

2.1.1 Canvas LMS

Table 2.1.1: Comparative Analysis of Academic LMS Platforms

Feature / Criterion	Moodle	Canvas LMS
Target Environment	Higher education, K-12, and enterprise training.	Primarily higher education and K-12.
System Architecture	Open-source. Highly modular, allowing deep database access and custom plugin development.	Proprietary (SaaS). Cloud-hosted with strict core codebase protection, relying on external LTI integrations.

(Continued on next page)

Table 2.1.1 (continued)

Feature / Criterion	Moodle	Canvas LMS
Course Registration	Managed via standard enrollment plugins or manual cohort syncs. Lacks native AI-driven prerequisite validation and advisement.	Integrates with university Student Information Systems (SIS). Does not feature an intelligent AI advisor for course selection.
Course & Document Management	Extensive file support. Instructors can upload syllabi and materials.	Excellent module-based organization and drag-and-drop file management.
Assessment & Grading	Comprehensive quizzing engine with rubrics and automated grading.	Highly intuitive "SpeedGrader" interface with built-in rubrics and annotation tools.
Early Warning & Analytics	Rule-based activity completion tracking and standard analytics models to flag inactive students.	Strong native learning analytics displaying student engagement, grade distribution, and interaction metrics.
AI Integration	No native AI out-of-the-box. However, its open architecture makes it the perfect foundation for building custom modules (like your document-grounded Q&A or risk-detection engines).	Relies on third-party integrations (e.g., external AI tutors via LTI). Cannot easily modify the core system to embed a native AI registration advisor.

(Continued on next page)

Table 2.1.1 (continued)

Feature / Criterion	Moodle	Canvas LMS
Role & User Management	Extremely granular Role-Based Access Control (RBAC). Admins can define custom permissions down to specific activity levels.	Standardized roles (Admin, Teacher, TA, Student) with solid, but less granular, permission customization.

Traditional LMS platforms like Moodle and Canvas function primarily as passive content repositories. The proposed system transitions into an active, AI-driven educational environment by natively embedding intelligence into university workflows. It surpasses standard systems through two core capabilities:

- **Contextual AI Assistant (RAG):** Embeds a Q&A chatbot directly grounded in uploaded course materials, eliminating reliance on fragmented third-party plugins.
- **Diagnostic Risk Detection:** Proactively identifies at-risk students and provides explicit causal explanations (e.g., consecutive scoring downtrends) rather than just displaying raw statistical charts.

2.1.2 D2L Brightspace

Table 2.1.2: Comparative Analysis of D2L Brightspace

Feature / Criterion	Moodle	D2L Brightspace
Target Environment	Supports higher education, K–12 education, and organizational training through a flexible learning management platform.	Supports higher education, K–12 education, and organizational learning through an integrated learning management platform.
System Architecture	Open-source and modular, allowing institutions to customize the platform and develop or install plugins.	Commercial LMS platform with integrated learning tools and additional analytics capabilities available through its product offerings.
Course Registration	Supports course enrollment through configurable enrollment methods, plugins, and administrative settings.	Supports course management and enrollment workflows. Its Student Success System focuses on predicting learner success rather than providing AI-based course recommendations.
Course & Document Management	Supports course creation, learning resources, activities, and file uploads through its course management features.	Supports course content organization, learning activities, and resources within the Brightspace learning environment.
Assessment & Grading	Provides quizzes, assignments, grading workflows, and configurable assessment activities.	Provides assessment and grading functions integrated with the learning environment. Its Student Success System also uses assessment-related indicators for predictive analysis.

(Continued on next page)

Table 2.1.2 (continued)

Feature / Criterion	Moodle	D2L Brightspace
Early Warning & Analytics	Provides activity completion tracking, reports, and learning analytics capabilities. Additional functionality can depend on installed plugins and institutional configuration.	The Student Success System uses predictive analytics and machine learning techniques to estimate learner success and predicted grades, helping instructors identify students who may need support.
AI Integration	Moodle's extensible architecture supports integration and customization through plugins and external services. Available AI capabilities depend on the implementation and installed tools.	The Student Success System integrates predictive analytics and machine learning into learner success monitoring, using historical course data to generate predictions and support early intervention.
Role & User Management	Supports role-based permissions for administrators, teachers, and students. Institutions can customize access and extend functionality through plugins and configuration.	Provides role-based access for administrators, instructors, and learners, with permissions managed through the Brightspace administrative environment.

Moodle and D2L Brightspace both provide core learning management capabilities, including course management, learning resources, and assessment support. Their approaches to customization and learning analytics differ. Moodle's open-source and modular architecture allows institutions to adapt the platform through plugins and integrations, while D2L Brightspace's Student Success System provides predictive analytics and machine learning capabilities to estimate learner success using historical course data.

This comparison highlights two relevant considerations for the project: the flexibility to integrate and customize LMS functionality, as demonstrated by Moodle, and the use of predictive analytics to support student monitoring, as demonstrated by D2L Brightspace. These considerations can inform the selection of the system architecture and the integration of AI capabilities according to the project's requirements.

2.2 Theory Foundation

2.2.1 Webserver

A web server is a software component that receives HTTP or HTTPS requests from clients and returns appropriate responses. It can deliver static resources, such as HTML, CSS, JavaScript, and images, or forward requests to an application server for further processing. Web servers can also support functions such as request routing, HTTPS termination, and reverse proxying, depending on their configuration.

2.2.2 Transaction Script pattern

Transaction script pattern organizes business logic by procedure where each procedure handles a single request from the presentation. Moodle mostly follow a transaction script approach. For example, if you open the Moodle forum page, the URL may be *.../mod/forum/view.php?id=1234* in which the *mod/forum/view.php* is the script that generates that page.

2.2.3 SQL

Structured Query Language is a standard, high-level, declarative language used to interact with relational database management systems (RDBMS). It allows users and applications to define database structures, retrieve and modify data, define views, specify access permissions and integrity constraints, and control transactions. As a non-procedural language, SQL specifies what data is required rather than how the database should process the request.

2.2.4 ERD (Chen Notation ERD)

The entity-relationship model represents the real world using entities, relationships, and their attributes. It can used as a basis for a unified view of data. An entity-relationship diagram is used to visualize this model. In this project, we will use the Chen notation ERD.

1. **Entities:** A thing or an object in the real world with an independent existence, being distinguishable.



Figure 2.1: Entity Type



Figure 2.2: Weak Entity Type

2. **Attributes:** A particular property that describes an entity via a value.

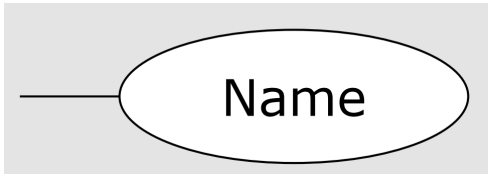


Figure 2.3: Attribute

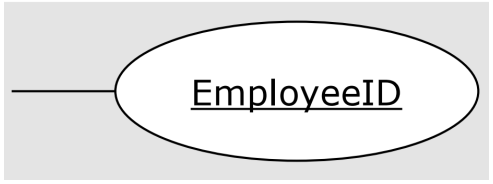


Figure 2.4: Key Attribute

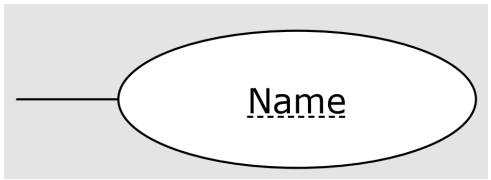


Figure 2.5: Partial Key Attribute

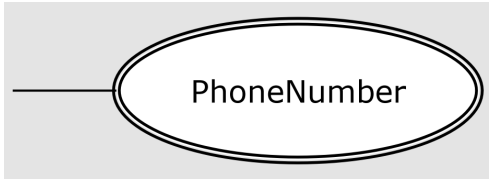


Figure 2.6: Multivalued Attribute

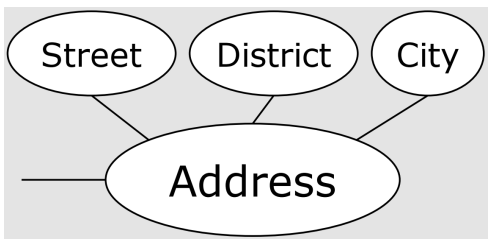


Figure 2.7: Composite Attribute



Figure 2.8: Devired Attribute

3. **Relationships:** A set of associations among entities.

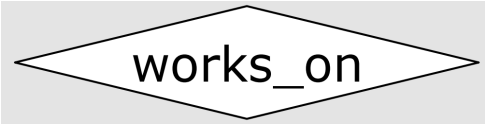


Figure 2.9: Relationship Type



Figure 2.10: Relationship Type of Weak Entity Type

4. **Structural Constraints:** Limitations and rules on how entities participate in relationships, determining valid data configurations.

   - **Single line:** Partial participation (optional).
   - **Double line:** Total participation (mandatory / existence dependency).
   - **1 and N:** Cardinality ratio (specifies 1:1, 1:N, or M:N associations).
   - **(min, max):** Explicit lower and upper bounds of participation.

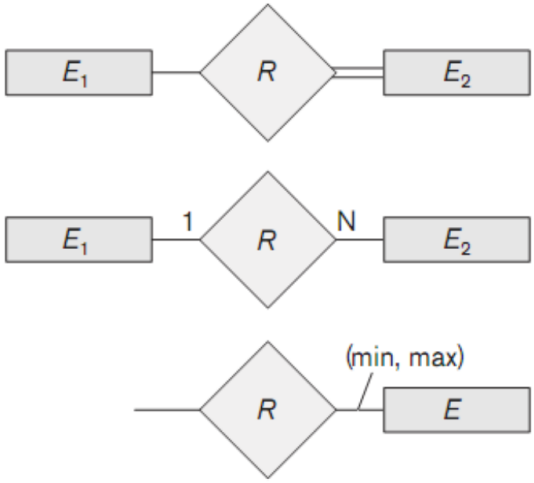


Figure 2.11: Structural Constraints on Relationship Types

2.3 Technologies

2.3.1 Python and FastAPI

Python is a high-level, general-purpose programming language known for its readable syntax, flexibility, and extensive ecosystem of libraries. It supports multiple program-

ming paradigms, including procedural, object-oriented, and asynchronous programming, making it suitable for developing modern backend applications and data-driven systems.
FastAPI is a modern Python web framework designed for building high-performance RESTful APIs. It uses Python type hints and Pydantic for request validation, data serialization, and schema definition. FastAPI also automatically generates interactive API documentation through OpenAPI and Swagger UI, simplifying API development, testing, and integration.
In this system, Python and FastAPI are used to implement backend business logic and expose API endpoints for communication between the client application, application services, and the database.

2.3.2 Uvicorn

Uvicorn is a lightweight ASGI (Asynchronous Server Gateway Interface) server designed to run asynchronous Python web applications. It uses an event-driven architecture to handle concurrent connections efficiently and supports asynchronous request processing through Python's `async` and `await` syntax.
Unlike a web framework, Uvicorn does not define application endpoints or business logic. Instead, it receives incoming HTTP requests, forwards them to the ASGI application, and returns the generated responses to clients.
In this system, Uvicorn serves as the application server for FastAPI, allowing the backend API to receive and process HTTP requests. It can be configured with multiple worker processes when additional request-processing capacity is required.

2.3.3 PostgreSQL

PostgreSQL is an open-source relational database management system that provides reliable data storage, advanced querying capabilities, and strong transactional guarantees. It supports SQL, primary and foreign keys, indexes, constraints, and database transactions to maintain data consistency and integrity.
PostgreSQL follows the ACID properties of database transactions: Atomicity, Consistency, Isolation, and Durability. These properties help ensure that related database operations are executed reliably. It also provides features such as indexing, concurrency

control, and extensible data types to support different application requirements.
In this system, PostgreSQL stores persistent application data, including user information, business records, and relationships between entities. Its transactional capabilities help maintain data integrity when multiple users or requests access the system concurrently.

2.3.4 Nginx

Nginx is a high-performance web server that can also function as a reverse proxy, load balancer, and HTTP cache. It is designed to handle a large number of concurrent connections with relatively low resource consumption, making it suitable for production environments.
As a reverse proxy, Nginx receives incoming client requests and forwards them to the appropriate backend application server. It can also handle HTTPS termination, configure request limits, and distribute traffic among multiple application instances. These capabilities help separate public-facing traffic management from backend application processing.
In this system, Nginx acts as the entry point for incoming HTTP or HTTPS requests and forwards API traffic to Uvicorn running the FastAPI application. This configuration provides a clear boundary between external clients and the backend services.

2.3.5 Docker

Docker is a containerization platform that packages an application and its dependencies into a standardized container image. Containers provide isolated execution environments while sharing the host operating system kernel, making them generally more lightweight than traditional virtual machines.
A Docker image defines the application environment, including the required runtime, libraries, configuration, and application files. A Docker container is a running instance of that image. Using a `Dockerfile` and container configuration allows developers to reproduce application environments consistently across development, testing, and deployment.
In this system, Docker packages the backend application and its runtime dependencies into containers. It can also be used to run PostgreSQL and Nginx as separate containers,

allowing the main components to be configured, deployed, and managed independently. Docker Compose can simplify local development by defining how these containers interact within a shared network.

2.3.6 PHP

(PHP: Hypertext Preprocessor) is a free, open-source, general-purpose scripting language particularly suited for web development. Its scripts execute on the server, enabling the creation of dynamic and interactive web pages, and PHP can be embedded into HTML.

2.3.7 Moodle

Moodle, short for Modular Object-Oriented Dynamic Learning Environment, is a free, open-source Learning Management System (LMS) designed to support online and blended learning. It provides a platform where educators can create and manage courses, share learning materials, organize activities, assess students, and support communication between learners and teachers. Since Moodle is open source, its code can be modified and extended to meet the needs of different educational institutions. Moodle provides a range of built-in learning tools and supports additional functionality through plugins and themes. It can be installed on a web server managed by an institution, allowing the institution to configure and customize the system according to its requirements. These capabilities make Moodle suitable as a foundation for projects that extend an existing learning environment with additional features.

User Interface. Moodle provides a web-based interface through which students, teachers, and administrators access learning and management functions. Depending on their roles and permissions, users can view courses, access learning resources, submit assignments, participate in activities, manage course content, and review grades. The appearance of a Moodle site can also be customized using themes, allowing institutions to adjust its layout and visual presentation.

Database. Moodle uses a database to store structured information required for the operation of the learning platform. This includes information about users, courses, enroll-

ments, activities, grades, and system configuration. Moodle supports several database systems, including PostgreSQL, MySQL-compatible databases, and Microsoft SQL Server. The database stores structured records, while uploaded files are handled through Moodle's file storage system.

Moodledata. The moodledata directory is used to store files and supporting data generated or managed during Moodle's operation. It includes uploaded course materials and user files, as well as temporary files, cached data, and session-related information. Unlike the database, which stores structured records, moodledata provides file storage for the Moodle site. For security, this directory should be located outside the web-accessible directory.

Moodle Architecture and Integration. Moodle consists of several components that work together to provide an online learning environment. The Moodle application contains the core functionality and handles user requests, while the database stores structured information and the moodledata directory stores uploaded files and other supporting data. These components work together to support course management, learning activities, and other platform functions.

Moodle also provides mechanisms for extending its functionality and communicating with external systems. Its plugin system allows developers to add features for specific purposes, while its Web Services framework enables external applications to access supported Moodle functions through configured endpoints. These mechanisms provide possible approaches for integrating additional services into an existing Moodle environment.

Regarding plugin organization, Moodle categorizes plugins by their types, with each type corresponding to a specific functionality and directory within the platform. For example, activity modules are stored in the `mod` directory, blocks are stored in the `blocks` directory, and local plugins are stored in the `local` directory. Moodle uses a naming convention known as *Frankenstyle* to uniquely identify plugin components. A component name consists of the plugin type prefix followed by the plugin name, separated by an underscore. For example, `mod_quiz` identifies the Quiz activity module, while `block_html` identifies the HTML block plugin. This convention provides a consistent way to identify plugins throughout the platform.

Moodle Community. Moodle is supported by an international community of educators,

administrators, developers, and organizations. Its official documentation, discussion forums, and plugin directory provide resources for learning, troubleshooting, and extending the platform. Developers can use Moodle's plugin system to add or customize functionality without necessarily changing the core code, making it easier to maintain extensions as the platform evolves.

2.3.8 Webpack

Webpack is a static module bundler for modern JavaScript applications. It analyzes an application's entry points and their dependencies to construct a dependency graph, then combines the required modules into one or more bundles that can be delivered to a web browser. Webpack supports different types of resources through loaders and plugins, allowing developers to configure how application files are processed during the build process.

2.3.9 React App

React is a JavaScript library for building user interfaces through reusable components. Each component represents a part of the interface and can contain its own presentation and behavior. By combining components, developers can construct larger interfaces while reusing common elements such as navigation bars, forms, buttons, tables, and dialog boxes. This component-based approach helps organize frontend code and makes interfaces easier to maintain as an application grows.

2.3.10 Generative AI

Generative Artificial Intelligence (Generative AI) refers to AI systems that generate new content in response to input instructions or contextual information. Depending on the model and application, generated content may include natural-language text, summaries, explanations, images, or other forms of output. Large Language Models (LLMs) are particularly relevant to text-based applications because they can interpret natural-language questions and produce responses in a conversational format.

2.3.11 GitHub

GitHub is a platform for hosting Git repositories and supporting collaborative software development. Git is a distributed version control system that records changes to files over time, while GitHub provides repository hosting and collaboration features around that version history. Developers can use repositories to organize source code and related project files, commits to record changes, and branches to work on separate features or fixes without directly modifying the main development branch.

Chapter 3

System Analysis

3.1 Functional Requirements

This section lists the functional requirements of the system, grouped by module. Because the project extends Moodle instead of rebuilding a learning management system, each requirement states whether it is **inherited** from Moodle (configured and exposed through the new portal) or **new** (developed by the team as plugins or external services). The *Phase* column indicates whether the requirement is delivered in the specialized project (Phase 1) or the graduation project (Phase 2).

3.1.1 Common user functions

Table 2.1.1: Functional requirements: common user functions

ID	Requirement	Source	Phase
FR-USR-01	The system shall allow every user to log in with a username or email and password and shall redirect the user to the dashboard of their role (Student, Teacher, or Administrator).	Inherited	1
FR-USR-02	The system shall allow users to log out and shall end a session automatically after a configurable period of inactivity.	Inherited	1

ID	Requirement	Source	Phase
FR-USR-03	The system shall allow users to view their profile and update editable fields (phone number, avatar, personal email), while keeping identity fields (user ID, full name, role) read-only.	Inherited	1
FR-USR-04	The system shall restrict every function and every piece of data according to the user's role and course enrollment (role-based access control).	New	1
FR-USR-05	The system shall display in-app notifications (e.g., new materials, deadlines, risk warnings) to the users concerned.	Inherited	1

3.1.2 Student module

Table 2.1.2: Functional requirements: student module

ID	Requirement	Source	Phase
FR-STU-01	The system shall allow students to browse, search, and filter the catalog of available courses and view course details (description, credits, prerequisites, lecturer, schedule).	Inherited	1
FR-STU-02	The system shall allow students to register for a course section during the registration period, checking prerequisites, credit limits, seat availability, and schedule conflicts.	Inherited	1
FR-STU-03	The system shall allow students to view the list of courses they are enrolled in.	Inherited	1
FR-STU-04	The system shall allow students to view and download the learning materials of their enrolled courses.	Inherited	1

ID	Requirement	Source	Phase
FR-STU-05	The system shall allow students to take online quizzes within the configured time window and attempt limit.	Inherited	1
FR-STU-06	The system shall allow students to submit assignments as files or online text before the deadline and shall mark late submissions.	Inherited	1
FR-STU-07	The system shall allow students to view their published grades and the feedback given by teachers.	Inherited	1
FR-STU-08	The system shall display each student's learning progress (completed and remaining activities) for each enrolled course.	Inherited	1
FR-STU-09	The system shall display a calendar of classes, assignment deadlines, and exam dates for the student's enrolled courses.	Inherited	1
FR-STU-10	The system shall allow students to communicate with teachers and classmates through course forums and messages.	Inherited	1

3.1.3 Teacher module

Table 2.1.3: Functional requirements: teacher module

ID	Requirement	Source	Phase
FR-TEA-01	The system shall allow teachers to view the list of courses they are assigned to teach and open each course's home page.	Inherited	1
FR-TEA-02	The system shall allow teachers to view, search, and filter the roster of enrolled students, including each student's last access time.	Inherited	1

ID	Requirement	Source	Phase
FR-TEA-03	The system shall allow teachers to upload learning materials (PDF, DOCX, PPTX, links) to course sections.	Inherited	1
FR-TEA-04	The system shall allow teachers to create, edit, and delete quizzes, including time windows, attempt limits, and questions from the question bank.	Inherited	1
FR-TEA-05	The system shall allow teachers to create, edit, and delete assignments with submission dates, deadlines, and file limits.	Inherited	1
FR-TEA-06	The system shall allow teachers to grade quizzes and assignments and give feedback to students.	Inherited	1
FR-TEA-07	The system shall allow teachers to view the academic performance of each student and of the whole class.	Inherited	1

3.1.4 Administrator module

Table 2.1.4: Functional requirements: administrator module

ID	Requirement	Source	Phase
FR-ADM-01	The system shall allow administrators to create, update, and deactivate user accounts, individually or by bulk import.	Inherited	1
FR-ADM-02	The system shall allow administrators to assign and revoke roles (Student, Teacher, Administrator).	Inherited	1
FR-ADM-03	The system shall allow administrators to create, update, and delete courses.	Inherited	1

ID	Requirement	Source	Phase
FR-ADM-04	The system shall allow administrators to create classes and sections, assign teachers to them, and assign students to them.	Inherited	1
FR-ADM-05	The system shall allow administrators to review course registrations and manage class schedules.	Inherited	1
FR-ADM-06	The system shall allow administrators to send notifications to selected users, classes, or all users.	Inherited	1
FR-ADM-07	The system shall allow administrators to manage system configuration, including the settings of the AI features (language model provider, risk thresholds, synchronization schedule).	New	1
FR-ADM-08	The system shall allow administrators to monitor the AI services, including request counts, latency, error rates, and language model token usage.	New	2

3.1.5 Data integration and document processing

Table 2.1.5: Functional requirements: data integration and document processing

ID	Requirement	Source	Phase
FR-INT-01	The system shall synchronize users, courses, enrollments, grades, submissions, and activity logs from Moodle to the extension services through Moodle Web Services or scheduled database extraction, without modifying the Moodle core.	New	1

ID	Requirement	Source	Phase
FR-INT-02	The system shall route every request from the portal to the AI Service through the backend gateway, which authenticates the user and checks permissions before forwarding the request.	New	1
FR-INT-03	The system shall automatically convert newly uploaded course materials (PDF, DOCX, PPTX) into structured Markdown text that keeps headings and page numbers.	New	1
FR-INT-04	The system shall split the converted text into chunks, compute their embeddings, and store them in the vector database together with metadata (course, section, file, page).	New	1
FR-INT-05	The system shall re-index a document when it is updated and remove its chunks when it is deleted from the course.	New	1
FR-INT-06	The system shall extract text from scanned documents using optical character recognition (OCR).	New	2

3.1.6 AI learning assistant (RAG chatbot)

Table 2.1.6: Functional requirements: AI learning assistant

ID	Requirement	Source	Phase
FR-CHAT-01	The system shall allow students and teachers to ask questions in natural language (Vietnamese or English) within the context of a course.	New	1
FR-CHAT-02	The system shall retrieve context only from the materials of courses the user is enrolled in or teaches.	New	1

ID	Requirement	Source	Phase
FR-CHAT-03	The system shall generate answers grounded in the retrieved context and attach citations (file name and page or section) to each answer.	New	1
FR-CHAT-04	The system shall state explicitly that it cannot find the answer when no sufficiently relevant context is retrieved, instead of generating unsupported content.	New	1
FR-CHAT-05	The system shall stream answers to the interface progressively as they are generated.	New	1
FR-CHAT-06	The system shall store chat sessions, messages, and citations so that users can review their conversation history and ask follow-up questions in context.	New	1
FR-CHAT-07	The system shall allow users to open the cited source document at the referenced page.	New	2
FR-CHAT-08	The system shall allow users to limit the search scope to a specific chapter or document and to rate answers (like or dislike).	New	2
FR-CHAT-09	The system shall combine semantic (vector) search with keyword (BM25) search and re-rank the retrieved chunks to improve answer accuracy.	New	2

3.1.7 Academic early warning

Table 2.1.7: Functional requirements: academic early warning

ID	Requirement	Source	Phase
FR-RISK-01	The system shall periodically compute risk indicators for each enrolled student, including days since last access, activity completion rate, recent assessment scores and their trend, submission delays, and number of missing submissions.	New	1
FR-RISK-02	The system shall flag at-risk students using configurable threshold rules (e.g., no access for more than 7 days, completion rate below 50%, latest score below 5.0 or missing submission).	New	1
FR-RISK-03	The system shall predict each student's probability of failing or withdrawing from a course with a machine learning model (gradient boosted trees) trained on historical learning data.	New	2
FR-RISK-04	The system shall classify each student into a risk level (High, Medium, or Low) based on the computed rules or predicted probability.	New	1
FR-RISK-05	The system shall explain each warning by listing its main contributing factors in plain language (e.g., "Inactive for 14 days", "Quiz score dropped 25% below the student's average"), computed with SHAP in Phase 2.	New	1
FR-RISK-06	The system shall display to teachers a list of at-risk students for each course, which can be sorted and filtered by risk level, together with a detailed view of each student's indicators and risk history.	New	1

ID	Requirement	Source	Phase
FR-RISK-07	The system shall display to each student only their own warnings, together with suggested actions (e.g., materials to review, assignments to submit).	New	1
FR-RISK-08	The system shall allow teachers to send a reminder message to an at-risk student, optionally starting from a draft generated by the language model from the warning factors.	New	2

3.1.8 Course recommendation

Table 2.1.8: Functional requirements: course recommendation

ID	Requirement	Source	Phase
FR-REC-01	The system shall recommend suitable courses to students based on their major, completed courses, prerequisites, and academic results, and shall give the reason for each recommendation.	New	2

3.2 Non-functional Requirements

The non-functional requirements below are stated with measurable targets wherever possible, so that they can be verified during testing. Because the system is evaluated at a small scale (a few courses and sample users, see Section 1.3), the targets are set for that scale and for the limited hardware available to the team.

3.2.1 Performance and scalability

Table 2.1.9: Non-functional requirements: performance and scalability

ID	Category	Requirement
NFR-PER-01	Response time	Non-AI pages and API calls of the portal shall respond within 2 seconds (95th percentile) with 100 concurrent users.
NFR-PER-02	AI response time	The chatbot shall display the first part of a streamed answer within 3 seconds and complete answers of up to about 300 words within 15 seconds (95th percentile).
NFR-PER-03	Dashboard load	The at-risk student dashboard of a class of up to 200 students shall load within 3 seconds, because risk scores are computed in advance and not at request time.
NFR-PER-04	Batch processing	Risk indicators and scores for all test courses shall be recomputed by a scheduled job (at least daily) that finishes within 30 minutes.
NFR-PER-05	Asynchronous processing	Document conversion, indexing, and model inference shall run asynchronously through a message queue, so that uploading a file or requesting an analysis never blocks the user interface; a 50-page PDF shall be searchable within 5 minutes of upload.
NFR-SCA-01	Scalability	The backend gateway and the AI Service shall be stateless and packaged as containers, so that more instances can be added behind the reverse proxy to handle a higher load without code changes.

3.2.2 Reliability and availability

Table 2.1.10: Non-functional requirements: reliability and availability

ID	Category	Requirement
NFR-REL-01	Fault isolation	A failure of the AI Service or the language model provider shall not affect the core LMS functions; the portal shall show a clear temporary error for the AI features only.
NFR-REL-02	Graceful degradation	When the warning model cannot run, the system shall display the most recently computed results together with their computation time, or fall back to the threshold rules.
NFR-REL-03	Data consistency	Data synchronization from Moodle shall be idempotent and resumable: re-running a synchronization job shall not create duplicate records.
NFR-REL-04	Backup	The databases (Moodle, application, and vector database) shall be backed up at least daily, and a backup shall be restorable within 2 hours.

3.2.3 Security and privacy

Table 2.1.11: Non-functional requirements: security and privacy

ID	Category	Requirement
NFR-SEC-01	Transport security	All communication between clients and the system, and between public services, shall use HTTPS (TLS 1.2 or higher).
NFR-SEC-02	Authentication	Every API request to the gateway and the AI Service shall carry a valid access token; tokens shall expire after a configurable period.

ID	Category	Requirement
NFR-SEC-03	Authorization	Role and enrollment checks shall be enforced on the server side for every request: a student can only access their own data and enrolled courses, and a teacher can only access the courses they teach.
NFR-SEC-04	Credential handling	User passwords shall be managed only by Moodle and never stored by the extension services; API keys and secrets shall be stored in environment variables or a secret store, never in source code.
NFR-SEC-05	Abuse prevention	The chatbot shall limit each user to a configurable number of requests per minute (default 20), validate all inputs, and apply instructions that prevent prompt injection from overriding system rules.
NFR-PRI-01	Data minimization	Only the question and the retrieved course text shall be sent to an external language model; names, student IDs, emails, and grades shall not be included in prompts, and student identifiers shall be pseudonymized.
NFR-PRI-02	Warning visibility	Risk warnings of a student shall be visible only to that student, the teachers of the course, and administrators.
NFR-AUD-01	Auditability	The system shall log administrative actions and AI requests (user, course, time, latency, token usage, status) for monitoring and auditing, without storing passwords or access tokens.

3.2.4 Maintainability, extensibility, and portability

Table 2.1.12: Non-functional requirements: maintainability, extensibility, and portability

ID	Category	Requirement
NFR-MNT-01	Upgrade safety	The Moodle core shall not be modified; all extensions shall be implemented as Moodle plugins or external services, so that Moodle can be upgraded independently.
NFR-MNT-02	Modularity	The portal, backend gateway, data-processing service, and AI Service shall communicate only through documented REST APIs (OpenAPI specification), so that each component can be developed, tested, and deployed independently.
NFR-MNT-03	Extensibility	The language model provider, embedding model, and warning model shall be replaceable through configuration and model versioning, without changing business logic.
NFR-MNT-04	Code quality	Source code shall be managed with Git using feature branches merged into a development branch through pull requests; core business logic and AI pipelines shall have automated tests.
NFR-POR-01	Portability	The whole system (Moodle, databases, backend gateway, AI Service, and portal) shall be started with a single Docker Compose configuration in development and test environments.

3.2.5 Usability and compatibility

Table 2.1.13: Non-functional requirements: usability and compatibility

ID	Category	Requirement
NFR-USA-01	Language	The interface shall support Vietnamese as the primary language and English as a secondary language.
NFR-USA-02	Clarity	Risk warnings shall be presented with a color-coded level (red, amber, green) and their contributing factors in plain language, so that users without technical knowledge can understand why a student was flagged.
NFR-USA-03	Responsiveness	The portal shall be usable on screens from 360 pixels wide (mobile phones) to desktop monitors.
NFR-USA-04	Learnability	A new user shall be able to ask the chatbot a question or open the at-risk student list without training, within three clicks from their dashboard.
NFR-COM-01	Browser support	The portal shall work on the two latest major versions of Google Chrome, Microsoft Edge, and Mozilla Firefox.
NFR-COM-02	Platform	The extensions shall be compatible with Moodle 5.x and PostgreSQL.

3.2.6 Quality of AI components

Table 2.1.14: Non-functional requirements: quality of AI components

ID	Category	Requirement
NFR-AI-01	Retrieval quality	On a test set of at least 100 questions built by the team from course materials, the relevant chunk shall be among the top 5 retrieved results (Recall@5) for at least 80% of the questions.

ID	Category	Requirement
NFR-AI-02	Answer faithfulness	Answers shall reach a faithfulness score of at least 0.8 (RAGAS), and every answer shall either cite at least one source or state that no answer was found.
NFR-AI-03	Warning effectiveness	Evaluated on held-out course presentations, the warning model shall reach a recall of at least 0.75 for the at-risk class when about 40% of the course has elapsed, and a higher PR-AUC than the logistic regression baseline.
NFR-AI-04	Explainability	Every risk prediction shall include its top three contributing factors, and the displayed factors shall be consistent with the model's SHAP values.
NFR-AI-05	Fairness	The difference in model performance between demographic groups (e.g., gender, region) shall be measured (ABROCA) and reported; demographic attributes shall not be used as input features.
NFR-AI-06	Human oversight	AI outputs shall be presented as decision support only; the system shall not automatically take academic decisions (such as grading or penalties) based on them.

3.3 Use case

3.3.1 System Actors

Table 2.1.15: System actors

Actor	Description
User	Abstract actor representing any authenticated person using the system. It owns the common use cases Login, Logout, and Update Profile, which are inherited by all concrete human actors.
Student	Learner enrolled in one or more courses. Browses and registers for courses, accesses learning materials, takes quizzes, submits assignments, tracks progress, asks the AI assistant questions about course materials, and views personal learning risk warnings.
Teacher	Lecturer assigned to teach courses. Manages course materials, quizzes, and assignments, monitors enrolled students, and reviews at-risk students together with the explanations produced by the warning model.
Administrator	Staff member responsible for operating the system. Manages user accounts and roles, courses, classes and schedules, notifications, and system configuration.
AI Service	Supporting system actor that answers course-related questions using Retrieval-Augmented Generation and computes academic risk scores with their explanations. It is invoked by the backend and never acts on its own initiative.

3.3.2 System use case diagram

System use case

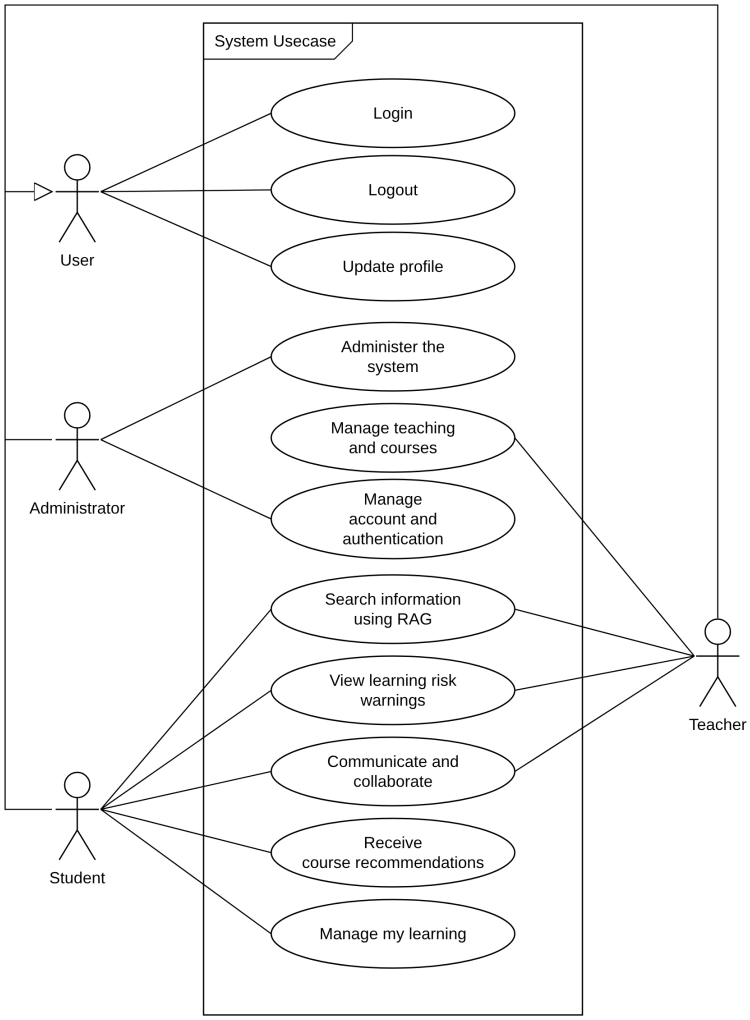


Figure 3.1: System Use Case Diagram

Use-case ID	UC-USR-01
Use-case name	Login
Author	Nguyen Manh Tri Minh
Description	Allows a user (Student, Teacher, or Administrator) to authenticate and access the features permitted by their role.
Actor	User (Student, Teacher, Administrator)
Trigger	The user opens the login page of the system portal.

Precondition	1. The user has a valid, active account in the system. 2. The user is not currently logged in.
Postcondition	The user is authenticated, a secure session (access token) is issued, and the user can access system features according to their role.
Normal Flow	1. The user enters their credentials (username or email and password). 2. The user submits the login form. 3. The system forwards the credentials to the authentication service and validates them against the user's account. 4. The system retrieves the user's role and permissions. 5. The system creates a secure session and issues an access token. 6. The system redirects the user to the dashboard corresponding to their role (student, teacher, or administrator dashboard).
Alternative Flow	**A1: Valid session already exists** 1. At step 1, the system detects a valid, unexpired session for the user. 2. The system skips the login form and redirects the user directly to their dashboard.

Exception Flow	**E1: Invalid credentials** 1. At step 3 of the normal flow, the credentials do not match any account. 2. The system rejects the login attempt, displays an error message, and prompts the user to try again. **E2: Account locked or suspended** 1. At step 3 of the normal flow, the system detects that the account is locked or suspended. 2. The system denies access and displays a message asking the user to contact the administrator. **E3: Authentication service unavailable** 1. At step 3 of the normal flow, the authentication service does not respond. 2. The system displays a temporary service error and allows the user to retry later.

Use-case ID	UC-USR-02
Use-case name	Logout
Author	Nguyen Manh Tri Minh
Description	Allows a user to securely terminate their active session and leave the system.
Actor	User (Student, Teacher, Administrator)
Trigger	The user selects the logout function from the navigation menu.
Precondition	The user is currently logged into the system.
Postcondition	The user’s session is terminated and the access token can no longer be used.

Normal Flow	1. The user clicks the logout button. 2. The system invalidates the active session and clears the authentication tokens stored on the client. 3. The system redirects the user to the login page.
Alternative Flow	**A1: Automatic logout after inactivity** 1. The user remains inactive for longer than the configured session timeout. 2. The system invalidates the session automatically. 3. On the next request, the system redirects the user to the login page.
Exception Flow	**E1: Session already expired** 1. At step 2 of the normal flow, the system detects that the session has already expired. 2. The system redirects the user to the login page without displaying an error.

Use-case ID	UC-USR-03
Use-case name	Update Profile
Author	Nguyen Manh Tri Minh
Description	Allows a user to view and update the editable fields of their personal profile, such as phone number, avatar, and personal email. Identity fields managed by the university (user ID, full name, role) are read-only.
Actor	User (Student, Teacher, Administrator)
Trigger	The user selects the option to view or edit their personal profile.
Precondition	The user is authenticated and logged into the system.
Postcondition	The updated profile information is saved and reflected throughout the system.

Normal Flow	1. The user opens the profile page. 2. The system displays the current profile information, with editable fields enabled and identity fields shown as read-only. 3. The user updates the editable fields (e.g., phone number, avatar, personal email). 4. The user submits the form. 5. The system validates the data formats and checks that no read-only field has been changed. 6. The system saves the updated information. 7. The system confirms that the profile has been updated successfully.
Alternative Flow	**A1: Cancel editing** 1. At step 3 or 4, the user selects “Cancel”. 2. The system discards the changes and displays the unchanged profile.
Exception Flow	**E1: Invalid data format** 1. At step 5 of the normal flow, a field contains malformed data (e.g., an invalid email address or an unsupported image format). 2. The system highlights the invalid field and requests correction; the profile is not updated. **E2: Save failure** 1. At step 6 of the normal flow, the system fails to save the changes. 2. The system displays an error message and keeps the previous profile information.

Student use case

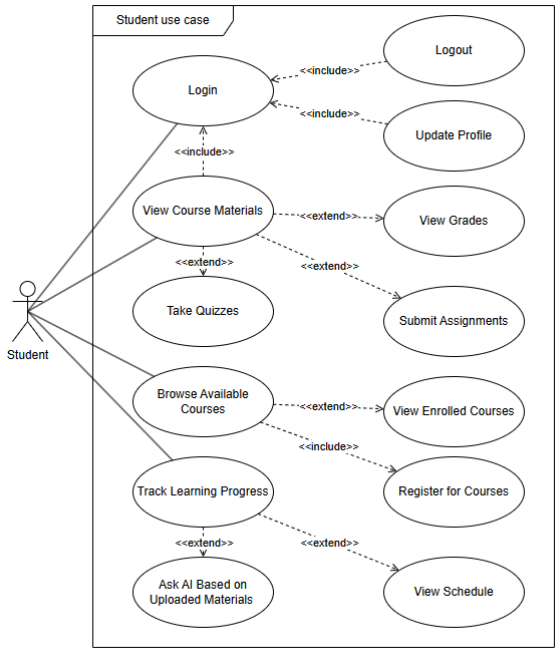


Figure 3.2: Student Use Case Diagram

Use-case ID	UC-STU-01
Use-case name	Login
Author	Dinh Cao Thien Loc
Description	Allows the student to authenticate and log into the system to access their personal dashboard and academic features.
Actor	Student
Trigger	The Student accesses the system's login portal.
Precondition	1. The Student has a valid, active account in the system. 2. The Student is not currently logged in.
Postcondition	The Student is authenticated and can access system features according to their role.

Normal Flow	1. The Student enters their credentials (e.g., username/email and password). 2. The Student submits the login form. 3. The system validates the entered credentials against the database. 4. If validation succeeds, the system authenticates the Student and generates a secure session. 5. The system redirects the Student to the student dashboard.
Alternative Flow	N/A.
Exception Flow	**E1: Invalid credentials** 1. At step 3 of the normal flow, the credentials do not match the database. 2. The system rejects the login attempt, displays an error message, and prompts the Student to try again. **E2: Account locked or suspended** 1. At step 3 of the normal flow, the system detects the account is locked. 2. The system denies access and displays a contact support message.

Use-case ID	UC-STU-02
Use-case name	Logout
Author	Dinh Cao Thien Loc
Description	Allows the student to securely terminate their active session and leave the system.
Actor	Student
Trigger	The Student selects the logout function from the navigation menu.
Precondition	The Student is currently logged into the system.

Postcondition	The Student's session is completely terminated.
Normal Flow	1. The Student clicks the logout button. 2. The system terminates the active user session and clears authentication tokens. 3. The system redirects the Student to the login page or public homepage.
Alternative Flow	N/A.
Exception Flow	**E1: Session already expired** 1. At step 2 of the normal flow, the system detects the session has already expired. 2. The system safely redirects the Student to the login page without error.

Use-case ID	UC-STU-03
Use-case name	Update Profile
Author	Dinh Cao Thien Loc
Description	Allows the student to modify and update allowable personal information such as phone number, avatar, and personal email.
Actor	Student
Trigger	The Student selects the option to edit their personal profile.
Precondition	The Student is authenticated and logged into the system.
Postcondition	The Student's updated profile information is saved and reflected in the system.

Normal Flow	1. The Student opens the profile management section. 2. The system displays the current profile information in an editable form. 3. The Student updates allowable fields (e.g., phone number, avatar, personal email). 4. The Student submits the updated form. 5. The system validates the data formats and checks for any prohibited changes. 6. The system saves the updated information to the database. 7. The system confirms that the profile has been updated successfully.
Alternative Flow	N/A.
Exception Flow	**E1: Invalid data format** 1. At step 5 of the normal flow, the user inputs malformed data (e.g., malformed email). 2. The system identifies the problem, highlights the field, and requests correction; the profile is not updated.

Use-case ID	UC-STU-04
Use-case name	Browse Available Courses
Author	Dinh Cao Thien Loc
Description	Allows the student to view, search, and filter the catalog of available courses for the upcoming or current semester.
Actor	Student
Trigger	The Student navigates to the course catalog or registration section.
Precondition	1. The Student is authenticated. 2. The system has active courses configured for the upcoming or current semester.

Postcondition	The Student successfully views the available courses and their details.
Normal Flow	1. The Student opens the course catalog interface. 2. The system retrieves the list of available courses and displays them. 3. The Student uses search filters (e.g., department, schedule, lecturer) to find specific courses. 4. The system updates the displayed list based on the selected filters. 5. The Student clicks on a specific course to view detailed information (description, credits, prerequisites).
Alternative Flow	N/A.
Exception Flow	**E1: No courses match the search filters** 1. At step 4 of the normal flow, no courses match the applied filters. 2. The system displays a “No courses found” message.

Use-case ID	UC-STU-05
Use-case name	Register for Courses
Author	Dinh Cao Thien Loc
Description	Allows the student to officially enroll in a specific course or section during the registration period.
Actor	Student
Trigger	The Student selects the function to register for a specific course/section.
Precondition	1. The Student is authenticated. 2. The course registration period is officially open.
Postcondition	The Student is officially enrolled in the course, and the available seat count is decreased by one.

Normal Flow	1. The Student views an available course and clicks the "Register" button. 2. The system checks the Student's eligibility (e.g., prerequisites passed, maximum credit limit not exceeded, schedule conflicts). 3. The system checks if there are available seats in the selected course section. 4. The system records the Student's enrollment in the course. 5. The system confirms the successful registration.
Alternative Flow	N/A.
Exception Flow	**E1: Prerequisites not met** 1. At step 2 of the normal flow, the student does not meet the course prerequisites. 2. The system denies registration and lists the missing prerequisite courses. **E2: Class is full** 1. At step 3 of the normal flow, there are no available seats. 2. The system denies registration and offers an option to join a waitlist (if applicable). **E3: Schedule conflict** 1. At step 2 of the normal flow, the system detects a scheduling conflict with existing enrollments. 2. The system highlights the overlapping courses and denies registration.

Use-case ID	UC-STU-06
Use-case name	View Enrolled Courses
Author	Dinh Cao Thien Loc

Description	Allows the student to view a list of all courses they are currently enrolled in via their dashboard.
Actor	Student
Trigger	The Student navigates to their personal dashboard or “My Courses” section.
Precondition	The Student is authenticated.
Postcondition	The Student views their current enrollments.
Normal Flow	1. The Student selects the “My Courses” tab. 2. The system queries the database for all active enrollments linked to the Student. 3. The system displays a list of the enrolled courses, including basic details (course name, lecturer, quick links to materials).
Alternative Flow	N/A.
Exception Flow	**E1: No enrolled courses** 1. At step 2 of the normal flow, the student has no active enrollments. 2. The system displays an empty state message encouraging them to register.

Use-case ID	UC-STU-07
Use-case name	View Course Materials
Author	Dinh Cao Thien Loc
Description	Allows the student to access, view, or download learning materials uploaded by the lecturer for a specific course.
Actor	Student
Trigger	The Student selects a specific course and navigates to the “Materials” or “Modules” section.

Precondition	1. The Student is authenticated and officially enrolled in the requested course. 2. The Lecturer has uploaded materials to the course.
Postcondition	The Student successfully accesses the learning material.
Normal Flow	1. The Student clicks on a document, video, or link provided in the course syllabus. 2. The system verifies the Student's access permissions for that specific material. 3. The system displays or downloads the requested material to the Student's device.
Alternative Flow	N/A.
Exception Flow	**E1: Material is locked or not yet published** 1. At step 2 of the normal flow, the material is restricted by date or condition. 2. The system denies access and shows the date/condition when it will be available.

Use-case ID	UC-STU-08
Use-case name	View Grades
Author	Dinh Cao Thien Loc
Description	Allows the student to view their assessment scores, cumulative GPA, and feedback provided by the lecturer.
Actor	Student
Trigger	The Student navigates to the "Grades" or "Academic Record" section.
Precondition	1. The Student is authenticated and enrolled in courses. 2. Grades have been published by the Lecturer or system.
Postcondition	The Student successfully views their academic performance.

Normal Flow	1. The Student selects a specific course to view detailed grades or views the cumulative GPA dashboard. 2. The system retrieves the published assessment scores (quizzes, assignments, midterms, finals) for that Student. 3. The system displays the grades, weightages, and any feedback provided by the Lecturer.
Alternative Flow	N/A.
Exception Flow	**E1: Grades are hidden or not yet published** 1. At step 2 of the normal flow, the requested grades are not yet published. 2. The system displays a “Pending” or “Not Available” status for those specific assessments.

Use-case ID	UC-STU-09
Use-case name	Take Quizzes
Author	Dinh Cao Thien Loc
Description	Allows the student to attempt and submit an active online quiz within a scheduled time frame.
Actor	Student
Trigger	The Student selects an active quiz to begin an attempt.
Precondition	1. The Student is authenticated and enrolled in the course. 2. An active quiz is available and open for submission within the scheduled time frame.
Postcondition	The quiz attempt is saved, timestamped, and queued for grading (or auto-graded).

Normal Flow	1. The Student clicks “Start Quiz”. 2. The system records the start time and displays the quiz questions. 3. The Student selects/enters answers for the questions. 4. The Student clicks “Submit Quiz”. 5. The system saves the answers, records the submission time, and closes the attempt. 6. The system auto-grades the objective questions (if configured) and provides a submission confirmation.
Alternative Flow	N/A.
Exception Flow	**E1: Time expires before submission** 1. During step 3 of the normal flow, the allocated quiz time runs out. 2. The system automatically forces a submission of the currently saved answers. **E2: Attempt limit reached** 1. At step 1 of the normal flow, the system detects the student has used all allowed attempts. 2. The system prevents the Student from starting a new attempt.

Use-case ID	UC-STU-10
Use-case name	Submit Assignments
Author	Dinh Cao Thien Loc
Description	Allows the student to upload file attachments or text submissions into an open assignment dropbox.
Actor	Student
Trigger	The Student navigates to an assignment and chooses to upload their work.

Precondition	1. The Student is authenticated and enrolled in the course. 2. An assignment dropbox is open and accepting submissions.
Postcondition	The assignment file is securely stored and available for the Lecturer to grade.
Normal Flow	1. The Student clicks “Submit Assignment”. 2. The system displays the file upload or text entry interface. 3. The Student uploads the required files or enters text. 4. The Student confirms and submits. 5. The system verifies file types and size limits. 6. The system saves the submission, timestamps it, and links it to the Student and assignment. 7. The system displays a success receipt.
Alternative Flow	N/A.
Exception Flow	**E1: Invalid file type or size exceeded** 1. At step 5 of the normal flow, the file does not meet constraints. 2. The system rejects the file and prompts the Student to upload a valid format. **E2: Past deadline** 1. At step 4 of the normal flow, the deadline has already passed. 2. The system either rejects the submission (if hard deadline) or marks it visually as “Late” (if soft deadline).

Use-case ID	UC-STU-11
Use-case name	Track Learning Progress
Author	Dinh Cao Thien Loc

Description	Allows the student to monitor their completion percentage and remaining tasks for a specific course.
Actor	Student
Trigger	The Student views the course homepage or dashboard.
Precondition	The Student is authenticated and enrolled in courses that utilize progress tracking.
Postcondition	The Student views an accurate reflection of their course completion status.
Normal Flow	1. The Student opens a specific course. 2. The system calculates the completion percentage based on viewed materials, completed quizzes, and submitted assignments. 3. The system displays a visual progress bar or checklist to the Student.
Alternative Flow	N/A.
Exception Flow	**E1: Course does not utilize completion tracking** 1. At step 2 of the normal flow, progress tracking is disabled by the lecturer. 2. The system hides the progress bar interface.

Use-case ID	UC-STU-12
Use-case name	View Schedule
Author	Dinh Cao Thien Loc
Description	Allows the student to view their academic timetable, including classes, assignment deadlines, and exam dates on a calendar interface.
Actor	Student
Trigger	The Student navigates to the “Schedule” or “Calendar” module.

Precondition	1. The Student is authenticated and enrolled in at least one course. 2. Classes and events are mapped to a timetable.
Postcondition	The Student successfully views their academic timetable and upcoming deadlines.
Normal Flow	1. The Student opens the calendar view (daily, weekly, or monthly). 2. The system retrieves class timings, assignment deadlines, and exam dates related to the Student's enrollments. 3. The system plots these events onto a visual calendar interface. 4. The Student clicks on a specific event to view location, time, and details.
Alternative Flow	N/A.
Exception Flow	**E1: No scheduled events** 1. At step 2 of the normal flow, there are no events in the selected timeframe. 2. The system displays an empty calendar grid for the selected time period.

Use-case ID	UC-STU-13
Use-case name	Ask AI Questions Based on Uploaded/Course Materials
Author	Dinh Cao Thien Loc
Description	Allows the student to ask questions and receive AI-generated answers grounded exclusively in the uploaded materials of the course.
Actor	Student
Trigger	The Student opens the AI learning assistant interface within a course and submits a question.

Precondition	1. The Student is authenticated and enrolled in the course. 2. The AI module is active, and course materials have been successfully indexed into the system's vector database.
Postcondition	The Student receives a contextual, accurate answer based strictly on the course materials.
Normal Flow	1. The Student types an academic question into the AI chat interface and presses send. 2. The system receives the query and retrieves relevant context exclusively from the specific course's uploaded materials (RAG approach). 3. The AI engine processes the question alongside the retrieved context to generate an accurate response. 4. The system displays the AI-generated answer to the Student, including citations/references to the specific uploaded course documents.
Alternative Flow	N/A.

Exception Flow	**E1: Question is unrelated to course material** 1. At step 3 of the normal flow, the AI finds no relevant context in the course materials. 2. The AI responds that it cannot find the answer in the provided course context. **E2: AI service timeout/unavailability** 1. At step 2 or 3 of the normal flow, the AI service fails to respond in time. 2. The system displays a temporary error message advising the Student to try again later. **E3: Safety/Policy violation in prompt** 1. At step 2 of the normal flow, the student's prompt violates safety filters. 2. The system blocks the query and displays a warning regarding acceptable use.

Admin use case

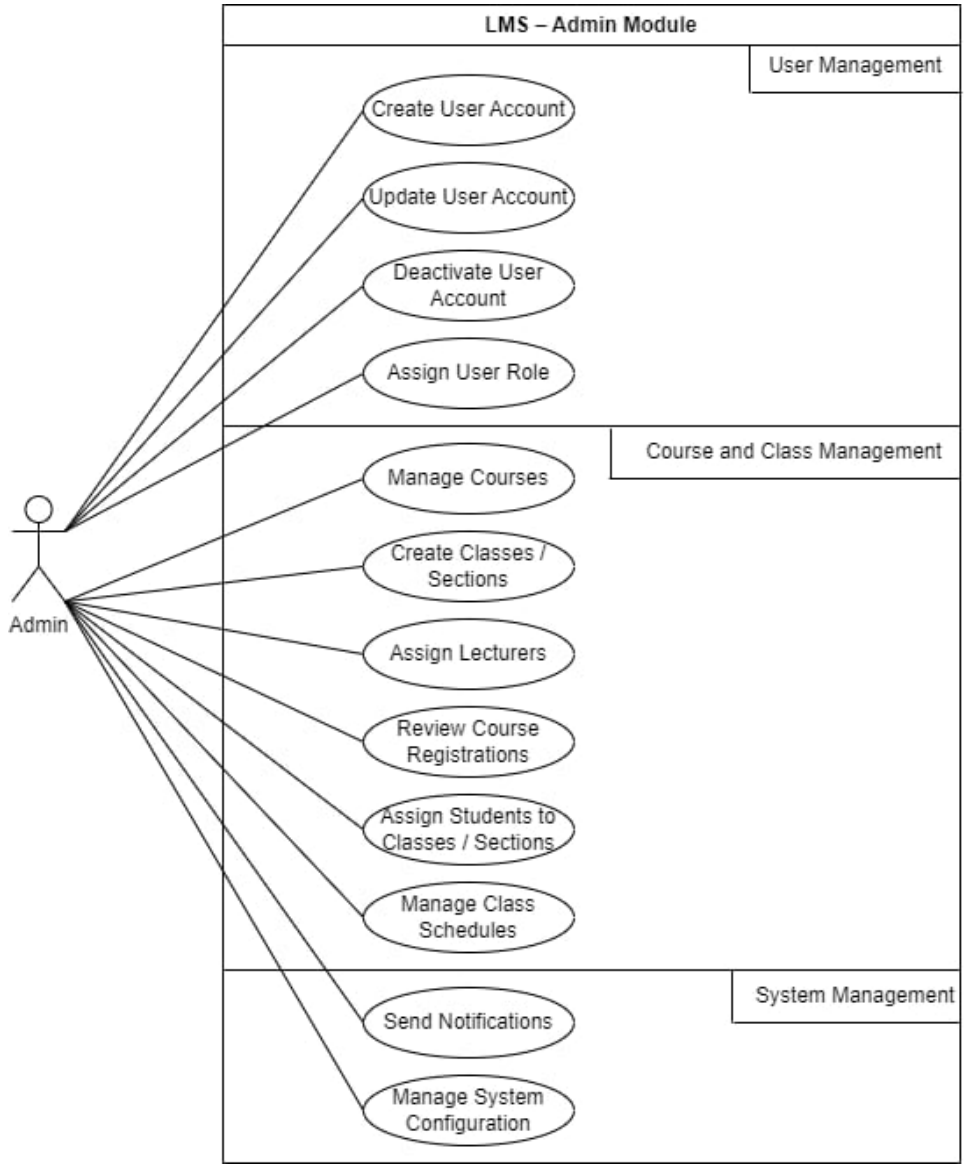


Figure 3.3: Admin Use Case Diagram

Use-case ID	UC-ADM-01
Use-case name	Create User Account
Author	[Author Name]
Description	Allows the administrator to create a new user account by entering the required information and assigning a mandatory role to the account.
Actor	System Administrator

Trigger	The System Administrator selects the create user account function.
Precondition	1. The System Administrator has a valid, active account. 2. The System Administrator has successfully logged in. 3. The system's user management service is available.
Postcondition	A new user account is created successfully with the assigned role and is stored in the system.
Normal Flow	1. The System Administrator opens the user management module. 2. The System Administrator selects the create user account function. 3. The system displays the user creation form. 4. The System Administrator enters the required user information. 5. The System Administrator selects a role for the new account: Student, Lecturer, or Admin. 6. The system validates the submitted information and checks whether the selected role is valid. 7. The system creates the new account and assigns the selected role. 8. The system displays a confirmation message.
Alternative Flow	**A1: Create an account with a different role** 1. At step 5, the System Administrator selects another available role. 2. The system updates the selected role in the creation form. 3. The System Administrator continues with step 6 of the normal flow.

Exception Flow	**E1: Missing required information** 1. At step 6 of the normal flow, the system detects missing required information. 2. The system displays validation errors and requests the System Administrator to complete the missing fields. **E2: Role not selected or invalid role** 1. At step 6 of the normal flow, the system detects that no role has been selected or the selected role is invalid. 2. The system rejects the submission and requests a valid role selection. **E3: Account already exists** 1. At step 6 of the normal flow, the system detects that the account information conflicts with an existing account. 2. The system displays an error message and requests different account information.

Use-case ID	UC-ADM-02
Use-case name	Update User Account
Author	[Author Name]
Description	Allows the administrator to view and update the information of an existing user account.
Actor	System Administrator
Trigger	The System Administrator selects a user account to update.
Precondition	1. The System Administrator has successfully logged in. 2. The System Administrator has the required administrative permissions. 3. The selected user account exists in the system.
Postcondition	The selected user account information is updated successfully, and the changes are stored in the system.

Normal Flow	1. The System Administrator opens the user management module. 2. The system displays the available user accounts. 3. The System Administrator searches for and selects a user account. 4. The system displays the selected account's information. 5. The System Administrator modifies the required information. 6. The system validates the updated information. 7. The system saves the changes. 8. The system displays a confirmation message.
Alternative Flow	**A1: View user account information** 1. At step 4, the System Administrator chooses to view the account without making changes. 2. The system displays the account information. 3. The use case ends without modifying the account.

Exception Flow	**E1: User account not found** 1. At step 3 of the normal flow, the system cannot find the selected account. 2. The system displays a notification that the account does not exist. **E2: Invalid account information** 1. At step 6 of the normal flow, the system detects missing or invalid information. 2. The system displays validation errors and requests corrections. **E3: Update failure** 1. At step 7 of the normal flow, the system cannot save the changes. 2. The system displays an error message and retains the previous account information.

Use-case ID	UC-ADM-03
Use-case name	Deactivate User Account
Author	[Author Name]
Description	Allows the administrator to deactivate an existing user account without permanently deleting it, preventing the user from logging in while preserving the account's data and learning history.
Actor	System Administrator
Trigger	The System Administrator selects the deactivate account function for a user.

Precondition	1. The System Administrator has successfully logged in. 2. The System Administrator has the required administrative permissions. 3. The selected user account exists and is currently active.
Postcondition	The selected account is deactivated, the user can no longer log in, and all associated data and learning history are preserved.
Normal Flow	1. The System Administrator opens the user management module. 2. The System Administrator searches for and selects a user account. 3. The system displays the account information and current status. 4. The System Administrator selects the deactivate account function. 5. The system requests confirmation of the operation. 6. The System Administrator confirms the deactivation. 7. The system changes the account status to inactive. 8. The system prevents the deactivated user from logging in. 9. The system preserves the account's associated data and learning history. 10. The system displays a confirmation message.
Alternative Flow	**A1: Cancel account deactivation** 1. At step 5, the System Administrator chooses to cancel the operation. 2. The system keeps the account active. 3. The use case ends without modifying the account status.

Exception Flow	**E1: Account not found** 1. At step 2 of the normal flow, the system cannot find the selected account. 2. The system displays an account not found message. **E2: Account already inactive** 1. At step 3 of the normal flow, the system detects that the account is already inactive. 2. The system informs the System Administrator that no deactivation is required. **E3: Deactivation failure** 1. At step 7 of the normal flow, the system cannot update the account status. 2. The system displays an error message and retains the previous account status.

Use-case ID	UC-ADM-04
Use-case name	Assign User Role
Author	[Author Name]
Description	Allows the administrator to assign or change a user's role among Student, Lecturer, and Admin according to the system's access control rules.
Actor	System Administrator
Trigger	The System Administrator selects the role assignment function for a user account.
Precondition	1. The System Administrator has successfully logged in. 2. The System Administrator has the required administrative permissions. 3. The selected user account exists in the system.

Postcondition	The selected user account is assigned the new valid role, and the role information is updated in the system.
Normal Flow	1. The System Administrator opens the user management module. 2. The System Administrator selects a user account. 3. The system displays the account information and current role. 4. The System Administrator selects the assign or change role function. 5. The system displays the available roles: Student, Lecturer, and Admin. 6. The System Administrator selects the required role. 7. The system validates the selected role and checks the administrator's permissions. 8. The system updates the user's role. 9. The system applies the corresponding access permissions. 10. The system displays a confirmation message.
Alternative Flow	**A1: Keep the current role** 1. At step 6, the System Administrator chooses to retain the existing role. 2. The system makes no changes to the account's role. 3. The use case ends.

Exception Flow	**E1: User account not found** 1. At step 2 of the normal flow, the system cannot find the selected account. 2. The system displays an account not found message. **E2: Invalid role selection** 1. At step 7 of the normal flow, the system detects a role that is not supported by the system. 2. The system rejects the role assignment and requests a valid selection. **E3: Role update failure** 1. At step 8 of the normal flow, the system cannot save the new role. 2. The system displays an error message and retains the previous role.

Use-case ID	UC-ADM-05
Use-case name	Manage Courses
Author	[Author Name]
Description	Allows the administrator to create, view, update, and deactivate courses in the learning management system.
Actor	System Administrator
Trigger	The System Administrator accesses the course management module.
Precondition	1. The System Administrator has successfully logged in. 2. The System Administrator has the required administrative permissions. 3. The course management service is available.

Postcondition	Course information is created or updated successfully, or the selected course is deactivated while its associated data is preserved.
Normal Flow	1. The System Administrator opens the course management module. 2. The system displays the available courses and management functions. 3. The System Administrator selects an operation, such as creating, viewing, updating, or deactivating a course. 4. The System Administrator enters or modifies the required course information. 5. The system validates the submitted information. 6. The system saves the new or updated course information, or changes the course status to inactive when deactivation is selected. 7. The system preserves associated course records when a course is deactivated. 8. The system displays a confirmation message.
Alternative Flow	**A1: View course information** 1. At step 3, the System Administrator selects a course to view. 2. The system displays the course details and current status. 3. The use case ends without modifying the course. **A2: Cancel course deactivation** 1. At step 3, the System Administrator selects the deactivation function but cancels the confirmation. 2. The system keeps the course active.

Exception Flow	**E1: Invalid course information** 1. At step 5 of the normal flow, the system detects missing or invalid course information. 2. The system displays validation errors and requests corrections. **E2: Course not found** 1. At step 3 of the normal flow, the system cannot find the selected course. 2. The system displays a course not found message. **E3: Course update failure** 1. At step 6 of the normal flow, the system cannot save the changes. 2. The system displays an error message and retains the previous course information and status.

Use-case ID	UC-ADM-06
Use-case name	Create Classes or Sections
Author	[Author Name]
Description	Allows the administrator to create a class or section associated with an existing course.
Actor	System Administrator
Trigger	The System Administrator selects the create class or section function.
Precondition	1. The System Administrator has successfully logged in. 2. The System Administrator has the required administrative permissions. 3. The associated course exists and is active.
Postcondition	A new class or section is created and associated with the selected course.

Normal Flow	1. The System Administrator opens the class management module. 2. The System Administrator selects the create class or section function. 3. The system displays the class creation form and available courses. 4. The System Administrator selects an existing course. 5. The System Administrator enters the required class or section information. 6. The system validates the submitted information and verifies the selected course. 7. The system creates the class or section and associates it with the selected course. 8. The system displays a confirmation message.
Alternative Flow	**A1: Select another course** 1. At step 4, the System Administrator selects a different available course. 2. The system updates the selected course in the form. 3. The System Administrator continues with step 5 of the normal flow.

Exception Flow	**E1: Course not found or inactive** 1. At step 6 of the normal flow, the system detects that the selected course does not exist or is inactive. 2. The system rejects the creation request and asks the System Administrator to select an active course. **E2: Invalid class information** 1. At step 6 of the normal flow, the system detects missing or invalid class information. 2. The system displays validation errors and requests corrections. **E3: Class creation failure** 1. At step 7 of the normal flow, the system cannot create the class or section. 2. The system displays an error message and allows the System Administrator to retry.

Use-case ID	UC-ADM-07
Use-case name	Assign Lecturers
Author	[Author Name]
Description	Allows the administrator to assign eligible lecturers to classes or courses while ensuring that the selected lecturer has the correct role and no scheduling conflicts.
Actor	System Administrator
Trigger	The System Administrator selects the lecturer assignment function for a class or course.

Precondition	1. The System Administrator has successfully logged in. 2. The selected course and class or section exist and are active. 3. At least one active lecturer account is available in the system.
Postcondition	An eligible lecturer is assigned to the selected class or course without violating the scheduling constraints.
Normal Flow	1. The System Administrator opens the class or course management module. 2. The System Administrator selects a class or course. 3. The System Administrator selects the assign lecturer function. 4. The system displays eligible lecturer accounts. 5. The System Administrator selects a lecturer. 6. The system verifies that the selected account is active and has the Lecturer role. 7. The system checks whether the lecturer is eligible to teach the selected course. 8. The system checks for scheduling conflicts involving the lecturer. 9. The system assigns the lecturer to the selected class or course. 10. The system displays a confirmation message.
Alternative Flow	**A1: Select another lecturer** 1. At step 5, the System Administrator selects a different lecturer. 2. The system performs the eligibility and scheduling checks for the newly selected lecturer. 3. The System Administrator continues with the assignment process.

Exception Flow	**E1: Invalid lecturer account** 1. At step 6 of the normal flow, the system detects that the selected account is inactive or does not have the Lecturer role. 2. The system rejects the assignment and requests another eligible lecturer. **E2: Lecturer not eligible for the course** 1. At step 7 of the normal flow, the system determines that the lecturer is not associated with or eligible to teach the selected course. 2. The system rejects the assignment and displays an appropriate message. **E3: Lecturer scheduling conflict** 1. At step 8 of the normal flow, the system detects a time conflict with another assigned class. 2. The system rejects the assignment and informs the System Administrator of the conflict. **E4: Assignment failure** 1. At step 9 of the normal flow, the system cannot save the lecturer assignment. 2. The system displays an error message and leaves the previous assignment unchanged.

Use-case ID	UC-ADM-08
Use-case name	Review Course Registrations
Author	[Author Name]
Description	Allows the administrator to review student course registration requests and approve or reject them.
Actor	System Administrator

Trigger	The System Administrator opens the course registration management module.
Precondition	1. The System Administrator has successfully logged in. 2. The System Administrator has the required administrative permissions. 3. Student course registration requests are available for review.
Postcondition	The selected registration request is approved or rejected, and its status is updated in the system.
Normal Flow	1. The System Administrator opens the course registration management module. 2. The system displays pending student registration requests. 3. The System Administrator selects a registration request. 4. The system displays the student's information and requested course. 5. The System Administrator reviews the registration information. 6. The System Administrator chooses to approve or reject the request. 7. The system validates the request and updates its registration status. 8. The system records the decision. 9. The system displays a confirmation message.

Alternative Flow	**A1: Approve a registration request** 1. At step 6, the System Administrator selects approve. 2. The system changes the registration status to approved. 3. The approved registration becomes eligible for the subsequent class assignment process. **A2: Reject a registration request** 1. At step 6, the System Administrator selects reject. 2. The system changes the registration status to rejected. 3. The system records the decision.
Exception Flow	**E1: Registration request not found** 1. At step 3 of the normal flow, the system cannot find the selected registration request. 2. The system displays a notification that the request is unavailable. **E2: Request already processed** 1. At step 7 of the normal flow, the system detects that the request has already been approved or rejected. 2. The system informs the System Administrator of the current request status. **E3: Registration update failure** 1. At step 7 of the normal flow, the system cannot update the registration status. 2. The system displays an error message and retains the previous status.

Use-case ID	UC-ADM-09
Use-case name	Assign Students to Classes or Sections
Author	[Author Name]

Description	Allows the administrator to assign eligible students to classes or sections after their course registration has been approved.
Actor	System Administrator
Trigger	The System Administrator selects the assign students function for a class or section.
Precondition	1. The System Administrator has successfully logged in. 2. The selected course and class or section exist and are active. 3. The student has an active account with the Student role. 4. The student's registration for the selected course has been approved.
Postcondition	The eligible student is assigned to the selected class or section without duplicate enrollment.
Normal Flow	1. The System Administrator opens the class management module. 2. The System Administrator selects a course and its class or section. 3. The System Administrator selects the assign students function. 4. The system displays students with approved registration for the selected course. 5. The System Administrator selects a student. 6. The system verifies that the student's account is active and has the Student role. 7. The system verifies that the student's course registration has been approved. 8. The system checks whether the student is already assigned to the selected class or section. 9. The system assigns the student to the class or section. 10. The system displays a confirmation message.

Alternative Flow	**A1: Assign multiple students** 1. At step 5, the System Administrator selects multiple eligible students. 2. The system validates each selected student's eligibility. 3. The system assigns the eligible students to the selected class or section. 4. The system displays the assignment results.
Exception Flow	**E1: Invalid student account** 1. At step 6 of the normal flow, the system detects that the selected account is inactive or does not have the Student role. 2. The system rejects the assignment. **E2: Course registration not approved** 1. At step 7 of the normal flow, the system detects that the student's registration has not been approved. 2. The system rejects the assignment and informs the System Administrator. **E3: Duplicate class assignment** 1. At step 8 of the normal flow, the system detects that the student is already assigned to the selected class or section. 2. The system prevents duplicate assignment and displays a notification. **E4: Assignment failure** 1. At step 9 of the normal flow, the system cannot save the assignment. 2. The system displays an error message and allows the System Administrator to retry.

Use-case ID	UC-ADM-10
Use-case name	Manage Class Schedules

Author	[Author Name]
Description	Allows the administrator to create and update class schedules while checking for conflicts involving time, room, lecturer, and assigned students.
Actor	System Administrator
Trigger	The System Administrator opens the class schedule management function.
Precondition	1. The System Administrator has successfully logged in. 2. The System Administrator has the required administrative permissions. 3. The selected class or section exists in the system.
Postcondition	The class schedule is created or updated successfully without any detected scheduling conflicts.
Normal Flow	1. The System Administrator opens the class schedule management module. 2. The System Administrator selects a class or section. 3. The system displays the current schedule, if available. 4. The System Administrator enters or modifies the schedule information, including time, room, and assigned lecturer. 5. The system validates the submitted schedule information. 6. The system checks for time conflicts involving the room, lecturer, and assigned students. 7. The system saves the schedule if no conflicts are detected. 8. The system displays a confirmation message.

Alternative Flow	**A1: View an existing schedule** 1. At step 3, the System Administrator chooses to view the current schedule without modifying it. 2. The system displays the schedule details. 3. The use case ends without changing the schedule. **A2: Modify an existing schedule** 1. At step 4, the System Administrator updates an existing schedule. 2. The system validates the updated information and performs the conflict checks. 3. The System Administrator continues with step 7 of the normal flow if no conflicts are found.
Exception Flow	**E1: Invalid schedule information** 1. At step 5 of the normal flow, the system detects missing or invalid schedule information. 2. The system displays validation errors and requests corrections. **E2: Scheduling conflict detected** 1. At step 6 of the normal flow, the system detects a conflict involving the schedule time, room, lecturer, or assigned students. 2. The system rejects the schedule change and identifies the detected conflict. **E3: Schedule update failure** 1. At step 7 of the normal flow, the system cannot save the schedule. 2. The system displays an error message and retains the previous valid schedule.

Use-case ID	UC-ADM-11
Use-case name	Send Notifications
Author	[Author Name]
Description	Allows the administrator to send notifications to relevant users through the system's available notification channels.
Actor	System Administrator
Trigger	The System Administrator selects the send notification function.
Precondition	1. The System Administrator has successfully logged in. 2. The System Administrator has the required administrative permissions. 3. The notification service is available.
Postcondition	The notification is created and submitted to the selected delivery channel or channels.

Normal Flow	1. The System Administrator opens the notification management module. 2. The System Administrator selects the send notification function. 3. The system displays the notification form. 4. The System Administrator enters the notification title and content. 5. The System Administrator selects the intended recipients. 6. The System Administrator selects the available notification channels, such as in-system notifications or email. 7. The system validates the notification content and recipient information. 8. The system submits the notification through the selected channels. 9. The system displays a confirmation message.
Alternative Flow	**A1: Send a notification to a specific group** 1. At step 5, the System Administrator selects a specific group of users, such as Students or Lecturers. 2. The system identifies the users belonging to the selected group. 3. The System Administrator continues with step 6 of the normal flow.

Exception Flow	**E1: Invalid notification content** 1. At step 7 of the normal flow, the system detects missing or invalid notification information. 2. The system displays validation errors and requests corrections. **E2: No valid recipients** 1. At step 7 of the normal flow, the system determines that no valid recipients are available. 2. The system informs the System Administrator and does not send the notification. **E3: Notification delivery failure** 1. At step 8 of the normal flow, the system cannot submit the notification through one or more selected channels. 2. The system reports the delivery failure and displays the available delivery status.

Use-case ID	UC-ADM-12
Use-case name	Manage System Configuration
Author	[Author Name]
Description	Allows the administrator to view and update configurable system settings according to the administrator's permissions.
Actor	System Administrator
Trigger	The System Administrator accesses the system configuration module.
Precondition	1. The System Administrator has successfully logged in. 2. The System Administrator has the required administrative permissions. 3. The system configuration service is available.

Postcondition	The selected system configuration settings are updated successfully, and the changes are stored in the system.
Normal Flow	1. The System Administrator opens the system configuration module. 2. The system displays the available configuration settings. 3. The System Administrator selects a setting to view or modify. 4. The system displays the current value of the selected setting. 5. The System Administrator enters the new configuration value. 6. The system validates the submitted value and checks the administrator's permissions. 7. The system saves the updated configuration. 8. The system records the change and displays a confirmation message.
Alternative Flow	**A1: View configuration settings** 1. At step 3, the System Administrator chooses to view a setting without modifying it. 2. The system displays the current configuration value. 3. The use case ends without changing the configuration.

Exception Flow	**E1: Invalid configuration value** 1. At step 6 of the normal flow, the system detects an invalid configuration value. 2. The system displays a validation error and requests a valid value. **E2: Insufficient permissions** 1. At step 6 of the normal flow, the system determines that the administrator does not have permission to modify the selected setting. 2. The system rejects the operation and displays an authorization error. **E3: Configuration update failure** 1. At step 7 of the normal flow, the system cannot save the configuration. 2. The system displays an error message and retains the previous valid configuration.

Teacher use case

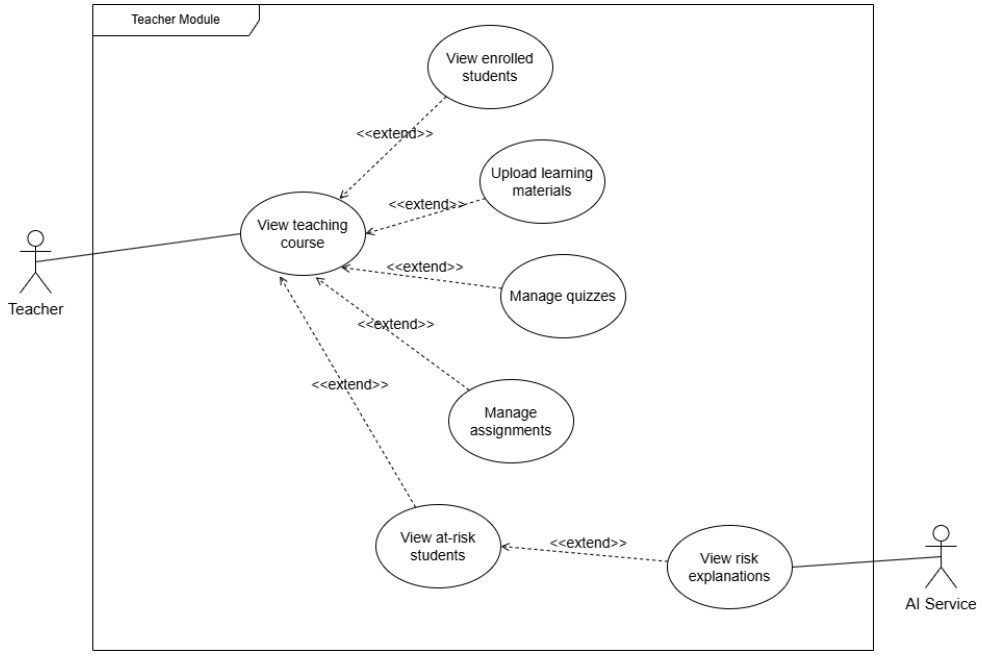


Figure 3.4: Teacher Use Case Diagram

Use-case ID	UC-010
Use-case name	View Teaching Course
Author	Thai Thien Hai Long
Description	Allows the lecturer to view the Course Home Page and its overview dashboard. It serves as the central entry point for accessing learning materials, assessments, student rosters, and AI risk analytics.
Actor	Teacher
Trigger	The teacher selects a specific course from their teaching course list on the dashboard.
Precondition	1. The user is authenticated and logged in with the teacher role. 2. The user has been assigned to teach at least one active course by the admin.

Postcondition	The course home page is displayed, presenting active course sections, announcements, and navigation links to course tools.
Normal Flow	1. The teacher navigates to the “My Courses” section from the main navigation menu. 2. The system retrieves and displays the list of courses assigned to the teacher. 3. The teacher selects a specific course to inspect. 4. The system navigates to the course home page. 5. The system displays course sections, announcements, and existing learning materials.
Alternative Flow	N/A.
Exception Flow	**E1: No Assigned Courses Found** 1. At step 2 of the normal flow, no courses are assigned to the teacher’s account. 2. The system displays a “No courses” placeholder. **E2: Course Load Failure** 1. At step 4 of the normal flow, the course fails to load due to a system error. 2. The system displays an error alert: “Unable to load course. Please try again.”

Use-case ID	UC-011
Use-case name	View Enrolled Students
Author	Thai Thien Hai Long
Description	Allows the teacher to view, filter, and monitor the enrolled student roster, including assigned groups and recent activity timestamps.
Actor	Teacher

Trigger	The teacher clicks on the “Participants” tab from the course navigation bar.
Precondition	1. The user is authenticated and logged in with the Teacher role. 2. The user has accessed the target course home page.
Postcondition	The enrolled student roster is displayed, including student IDs, full names, assigned groups, and last access timestamps.
Normal Flow	1. From the course home page, the teacher selects the “Participants” tab. 2. The system queries the database for all enrolled users associated with the course. 3. The system displays the participant table containing student names, ID numbers, email addresses, assigned groups, roles, and last access timestamps.
Alternative Flow	**A1: Filter or Search Students** 1. At step 3 of the normal flow, the teacher enters a search keyword or selects a class group filter. 2. The system filters and displays the matching subset of enrolled students.
Exception Flow	**E1: No Students Enrolled** 1. At step 2 of the normal flow, no student records are linked to the course. 2. The system displays an empty-state message: “No participants found in this course.” **E2: Participant Data Load Failure** 1. At step 2 of the normal flow, the participant list fails to load due to a database connection error. 2. The system displays an error alert: “Unable to retrieve participant list. Please refresh the page.”

Use-case ID	UC-012
Use-case name	Upload Learning Materials
Author	Thai Thien Hai Long
Description	Allows the teacher to add learning resources (e.g., lecture slides, documents, syllabus, or links) to specific course sections for enrolled students to access.
Actor	Teacher
Trigger	The teacher enables edit mode and selects “Add an activity or resource” within a course section.
Precondition	1. The user is authenticated and logged in with the Teacher role. 2. The user has accessed the target course home page.
Postcondition	The uploaded learning material is saved and displayed within the selected course section.
Normal Flow	1. From the course home page, the teacher turns on “Edit mode”. 2. In the target section, the teacher clicks “Add an activity or resource” and selects “File” (or “Folder”). 3. The system displays the resource setup form. 4. The teacher inputs the resource name, description, and uploads the file attachment. 5. The teacher clicks “Save and return to course”. 6. The system validates the file, stores it, and displays the new resource link in the course section.

Alternative Flow	**A1: Upload via Drag-and-Drop** 1. At step 2 of the normal flow, the teacher directly drags and drops a file from the local file explorer into the desired section on the course page. 2. The system automatically uploads the file and creates a resource entry using the filename. **A2: Add External URL Resource** 1. At step 2 of the normal flow, the teacher selects "URL" instead of "File". 2. The teacher enters the resource title and external web address. 3. The flow proceeds to step 5.
Exception Flow	**E1: File Size Exceeds Limit** 1. At step 5 of the normal flow, the uploaded file exceeds the course upload size limit. 2. The system rejects the file and displays an alert: "The file is too large to be uploaded." 3. The flow returns to step 4 for file re-selection. **E2: Unsupported File Format** 1. At step 5 of the normal flow, the file format is restricted by system policy. 2. The system displays an error: "This file type is not allowed." 3. The flow returns to step 4.

Use-case ID	UC-013
Use-case name	Manage Quizzes
Author	Thai Thien Hai Long

Description	Allows the teacher to create, configure, update, or remove online quiz assessments and define question sets and grading rules for the course.
Actor	Teacher
Trigger	The teacher enables edit mode and selects “Add an activity or resource” > “Quiz” within a course section.
Precondition	1. The user is authenticated and logged in with the Teacher role. 2. The user has accessed the target course home page.
Postcondition	The quiz activity and its configured question items are saved and published in the designated course section.
Normal Flow	1. From the course home page, the teacher turns on “Edit mode”. 2. In the target section, the teacher clicks “Add an activity or resource” and selects “Quiz”. 3. The system displays the quiz settings page (timing, grade boundaries, attempt limits). 4. The teacher configures the quiz parameters and clicks “Save and display”. 5. The system opens the quiz question management interface. 6. The teacher adds questions from the question bank or creates new questions with point values. 7. The teacher saves the assessment. 8. The system updates the quiz structure and makes it accessible in the course section.

Alternative Flow	**A1: Edit Existing Quiz** 1. At step 2 of the normal flow, the teacher clicks the action menu of an existing quiz and selects “Edit settings”. 2. The system loads existing configurations and questions. 3. The teacher updates parameters or modifies questions. 4. The flow proceeds to step 7. **A2: Delete Quiz** 1. At step 2 of the normal flow, the teacher selects “Delete” on an existing quiz. 2. The system requests confirmation. 3. The teacher confirms deletion, and the system removes the quiz activity from the course.
Exception Flow	**E1: Invalid Time Window** 1. At step 4 of the normal flow, the quiz close time is configured before the open time. 2. The system displays a validation error: “The closing time cannot be earlier than the opening time.” 3. The teacher adjusts the timestamps. **E2: Empty Question Set** 1. At step 7 of the normal flow, the teacher attempts to publish a quiz without adding any questions. 2. The system displays a warning notice: “No questions have been added yet.”

Use-case ID	UC-014
Use-case name	Manage Assignments
Author	Thai Thien Hai Long

Description	Allows the teacher to create, configure, update, or remove student assignment dropboxes, specifying submission formats, deadlines, and grading criteria.
Actor	Teacher
Trigger	The teacher enables edit mode and selects “Add an activity or resource” > “Assignment” within a course section.
Precondition	1. The user is authenticated and logged in with the Teacher role. 2. The user has accessed the target course home page.
Postcondition	The assignment entry is created or updated, providing an active submission portal for enrolled students.
Normal Flow	1. From the course home page, the teacher turns on “Edit mode”. 2. In the target section, the teacher clicks “Add an activity or resource” and selects “Assignment”. 3. The system displays the assignment configuration form. 4. The teacher specifies assignment title, description instructions, and optional prompt attachments. 5. The teacher configures availability dates (allow submissions from, due date, cutoff date) and file submission limits. 6. The teacher clicks “Save and return to course”. 7. The system validates the submission parameters and publishes the assignment item.

Alternative Flow	**A1: Edit Existing Assignment** 1. At step 2 of the normal flow, the teacher clicks the action menu of an existing assignment and selects “Edit settings”. 2. The system populates the form with existing assignment configurations. 3. The teacher updates instructions or extends deadlines. 4. The flow proceeds to step 6. **A2: Delete Assignment** 1. At step 2 of the normal flow, the teacher selects “Delete” on an existing assignment. 2. The system prompts a confirmation dialog. 3. The teacher confirms deletion, and the system removes the assignment.
Exception Flow	**E1: Invalid Due Date** 1. At step 6 of the normal flow, the specified due date precedes the allowable submission start date. 2. The system halts submission and displays an error: “Due date must be after the allow submissions from date.” 3. The flow returns to step 5. **E2: File Size Exceeds Limit** 1. At step 4 of the normal flow, the attached assignment prompt file exceeds maximum upload size. 2. The system rejects the file and displays an error alert: “The uploaded instruction file exceeds size limits.”

Use-case ID	UC-015
Use-case name	View At-Risk Students and Risk Explanations
Author	Thai Thien Hai Long

Description	Allows the teacher to review an AI-analyzed roster of enrolled students at academic risk within a specific course, inspect feature-based explanations (e.g., inactivity duration, assignment delays, or quiz drop-offs), and trigger personalized outreach interventions.
Actor	Teacher, AI Service (Secondary)
Trigger	The teacher navigates to the “Analytics / At-Risk Students” section from the course management dashboard.
Precondition	1. The user is authenticated and logged in with the Teacher role. 2. The user has accessed a course they are actively assigned to teach. 3. The background feature extraction and AI prediction pipeline has processed current student activity and grade logs.
Postcondition	The teacher reviews categorized student risk profiles and explanations, and optional intervention actions (e.g., outreach email logs) are recorded.

Normal Flow	1. From the course home page, the teacher navigates to the "At-Risk Students" analytics overview. 2. The system queries the AI service and displays the student roster categorized into risk levels (*High Risk*, *Medium Risk*, *Safe*). 3. The teacher filters or sorts the roster by risk severity or student identifier. 4. The teacher selects a flagged student to view detailed risk indicators. 5. The system requests the explanation payload from the AI service. 6. The AI service computes/retrieves feature attributions and returns plain-language pedagogical reasons (e.g., prolonged inactivity, negative quiz score slopes, missing submissions). 7. The system renders the student risk profile along with the primary contributing factors and impact weights. 8. The teacher reviews the assessment and closes or acknowledges the report.

Alternative Flow	**A1: Generate AI Intervention Outreach** 1. At step 7 of the normal flow, the teacher clicks “Draft Personalized Outreach”. 2. The system prompts the generative AI service with the student’s contributing risk factors to draft an empathetic email. 3. The teacher reviews, edits, and sends the drafted communication. 4. The system logs the intervention status and returns to step 3. **A2: Filter by Safe or Moderate Thresholds** 1. At step 3 of the normal flow, the teacher selects a filter to inspect stable or low-risk students. 2. The system updates the roster view to show students whose performance indicators meet course baselines.
Exception Flow	**E1: Insufficient Activity Data for Evaluation** 1. At step 2 of the normal flow, the course has insufficient student interaction logs or submission records to compute reliable risk predictions. 2. The system displays an informational status banner: “Insufficient activity data available to generate early warning predictions.” 3. The flow terminates without rendering risk tiers. **E2: AI Service / Analytics Endpoint Unavailable** 1. At step 2 or step 5 of the normal flow, the connection to the AI analytics microservice times out or returns an error. 2. The system displays a notification alert: “Unable to retrieve predictive analytics. Please try again later.” 3. The system falls back to displaying raw gradebook summaries without AI risk classifications.

Chapter 4

Conclusion